

Model-aware prompting for multi-model agent systems

The single most impactful change for a multi-model agent system is not prompt wording—it's dynamic tool loading. Reducing the active tool set from 40 to 5–10 per query improves accuracy by **25+ percentage points** across all model families and eliminates the primary failure mode (wrong tool selection) that plagues large tool sets. Beyond tool management, a layered prompt system with model-family overlays—handling format preference (XML vs. Markdown), reasoning model detection (removing CoT instructions), and parallel calling guidance—captures roughly 80% of the gains achievable through model-specific optimization, without maintaining fully separate prompt variants.

This report synthesizes official vendor guidance from Anthropic, OpenAI, Google, and Meta/community sources, along with benchmarks from BFCL, ToolBench, TaskBench, MCPVerse, and practitioner studies. The findings map directly to a YAML-configured agent system where swapping the `(model)` field should produce near-optimal behavior for each backend.

Tool count is the dominant variable, not description quality

The research is unambiguous: **tool count drives accuracy more than any other factor**. TaskBench (NeurIPS 2024) demonstrates catastrophic degradation—**96% accuracy with 1–2 tools drops to 25% with 8 tools**.

`(NeurIPS)` The "Less is More" paper (`(arXiv)` `(arxiv)` 2411.15399) showed Llama-3.1-8B failing entirely with 46 tools but succeeding with 19. `(arXiv)` `(arxiv)` MCPVerse (`(arXiv)` `(arxiv)` 2508.16260) found that even frontier models degrade with 200+ tools, though Claude Sonnet 4 was the notable exception, actually improving accuracy with expanded tool sets. `(arXiv)`

The vendor-specific guidance converges on a clear recommendation:

- **Anthropic:** Use the Tool Search Tool when definitions exceed **10K tokens or 10+ tools**. Internal testing showed Opus 4 improving from 49% → 74% and Opus 4.5 from 79.5% → 88.1% with dynamic loading enabled. `(Anthropic)` Token savings: ~85% `(Anthropic)` (from 72K+ to ~3.5K for tool overhead). `(anthropic)`
- **OpenAI:** Fewer than ~100 tools and ~20 arguments per tool is "in-distribution" for o3/o4-mini, `(openai)` but description clarity becomes critical `(openai)` as count rises. Tool hallucinations increase with large, under-defined tool sets.
- **Google:** Explicitly recommends keeping the active set to **10–20 tools maximum** and suggests "dynamic tool selection based on conversation context" for larger registries. `(Google AI)` Hard API limit is 128 function declarations. `(Firebase)`
- **Small models (<70B):** Keep to **fewer than 10 tools per call** for reliable results. MCP-Bench shows Llama-3.1-8B scoring 0.438 in single-server settings, dropping to 0.415 with multiple tool servers. `(arXiv)`

For your system with 20–40 tools, the optimal architecture is a two-tier approach: **3–5 always-loaded core tools** plus a tool search mechanism that loads **5–10 relevant tools on demand** per query. This matches both the Anthropic Tool Search Tool pattern `(Anthropic)` and the "Dynamic ReAct" architecture (`(arXiv)` `(arxiv)` 2509.20386),

which found that LLM-constructed search queries into a vector database achieved the best accuracy across all tested approaches.

What each vendor actually recommends for agentic prompting

Claude: structured XML, extremely detailed descriptions, and reduced over-prompting

Anthropic's guidance is the most explicit about tool descriptions. The official docs state: **"Provide extremely detailed descriptions. This is by far the most important factor in tool performance. Aim for at least 3–4 sentences per tool description, more if the tool is complex."** [Claude API Docs](#) [Anthropic](#) Descriptions should cover what the tool does, when to use it (and when not to), what each parameter means, and important caveats.

[Claude API Docs](#)

For system prompt structure, **XML tags are the recommended format** (`<instructions>`, `<context>`), `<use_parallel_tool_calls>`). [anthropic](#) [Algorithm Unmasked](#) Role assignment goes in the system prompt; detailed instructions can go in the first user turn. [Anthropic](#) A critical nuance for Opus 4.5/4.6: these models are significantly more responsive to system prompts than predecessors. Instructions like "CRITICAL: You MUST use this tool" now cause over-triggering—replace with "Use this tool when it would enhance your understanding of the problem." [Anthropic](#) [anthropic](#)

Claude 4.x models excel at parallel tool execution, with Sonnet 4.5 being "particularly aggressive" at firing simultaneous calls. [anthropic](#) The official recommended prompt block uses XML:

```
xml

<use_parallel_tool_calls>
If you intend to call multiple tools and there are no dependencies between
the tool calls, make all of the independent tool calls in parallel.
</use_parallel_tool_calls>
```

Across the Claude tier—**Opus** handles complex multi-tool orchestration with the best accuracy (seeks clarification on ambiguous queries), **Sonnet** is the best all-rounder with aggressive parallelism, and **Haiku** should be limited to straightforward tasks (may infer missing parameters, loses context in long sessions).

[Claude API Docs](#)

OpenAI: prescriptive function descriptions, minimal CoT for reasoning models

OpenAI's o-series guidance contains one of the most counterintuitive findings: **asking a reasoning model to reason more can actually hurt performance**. The o3/o4-mini cookbook explicitly states: "A developer should not try to induce additional reasoning before each function call by asking the model to plan more extensively."

[OpenAI Cookbook](#) [openai](#) These models are trained with tools in their chain of thought—additional planning prompts interfere with their internal reasoning. [OpenAI Cookbook](#)

For function descriptions, OpenAI emphasizes structure over length. A study from their cookbook showed that **descriptions with key rules placed first scored 6% higher** than those with the same rules buried at the bottom. [OpenAI Cookbook](#) [openai](#) The critical principle: be prescriptive ("Always escape special regex characters"), not descriptive ("This tool performs fast regex-based text searches").

GPT-5 follows instructions with "surgical precision," which means contradictory or vague prompts cause more damage than with GPT-4o—the model expends reasoning tokens trying to reconcile contradictions. (openai) The GPT-5 prompting guide recommends the Responses API with (previous_response_id), which improved Tau-Bench scores from **73.9% to 78.2%** by preserving reasoning state across tool calls. (openai)

For parallel calling, OpenAI recommends disabling (parallel_tool_calls) when strict schema adherence is critical, as "model outputs may not match strict schemas supplied in tools" during parallel calls.

(OpenAI Developer Community) (Advanced-stack) The nano model (gpt-4.1-nano) specifically has issues with duplicate tool calls when parallel calling is enabled. (OpenAI)

Gemini: benchmark-tested system instruction templates, 10–20 tool ceiling

Google provides the most empirically grounded system instruction guidance. Their agentic system instruction template—structured around reasoning/strategy, execution/reliability, and interaction/output dimensions—**improved performance on several agentic benchmarks by approximately 5%**. (google +2) The template uses XML tags and is organized by behavioral dimensions:

```
<role>You are Gemini 3, a specialized assistant for [domain].</role>
<instructions>
1. Plan: Analyze the task and create a step-by-step plan.
2. Execute: Carry out the plan.
3. Validate: Review your output against the user's task.
</instructions>
```

Gemini 3 requires fundamentally different prompting than Gemini 2.5. It "responds best to direct, clear instructions" and "may over-analyze verbose or overly complex prompt engineering techniques used for older models." (Google AI) Temperature should stay at 1.0 (below 1.0 causes looping or degraded performance in reasoning tasks). Gemini 2.5 should use low temperature (~0) for deterministic function calls. (Google AI) (Databricks)

A critical implementation detail for Gemini: **thought signatures** must be returned across multi-turn function calling conversations. Gemini 3 enforces this strictly—missing signatures produce a 400 error. (Google AI) This is handled automatically by Google's official SDKs but requires explicit management in custom implementations. (Google AI) (Google AI)

OSS reasoning models: disable system prompts for R1, use Qwen3 for best tool calling

DeepSeek R1 does not natively support function calling (Together AI) in the official API (only DeepSeek-V3/Chat does). (Data Studios) (Ollama) For agentic use, R1 requires either ReAct-style prompting, (Data Leads Future) a community-maintained chat template with special tokens, (Hugging Face) or routing R1 for reasoning-only tasks while using V3 for tool execution. R1 also **ignores system prompts**—all instructions must go in the user role. (Together AI) No few-shot examples (they consistently degrade performance). Temperature should be 0.6 with top_p 0.95. (Together AI) (together)

Qwen3 offers the most sophisticated reasoning budget control among OSS models: hard switch ((enable_thinking=True/False)), (Hugging Face) soft per-turn toggles ((/think) and (/no_think) inline), (Hugging Face) and an explicit (thinking_budget) parameter. (arXiv) For tool calling, Qwen uses Hermes-style templates with (<tools>)

and `<tool_call>` XML tags. [Qwen](#) [GitHub](#) The Qwen team recommends against using vLLM's auto tool choice parser for reasoning models (QwQ/Qwen3)—use Qwen-Agent framework instead to handle the interaction between thinking tags and tool call parsing. [Hugging Face](#) [PyPI](#)

Llama 8B "cannot reliably maintain a conversation alongside tool-calling definitions" per Meta's guidance. [vLLM](#) Use 70B+ for reliable multi-tool conversations. [Substack](#) [Medium](#) A surprising BFCL finding: **prompt-based function calling often outperforms native FC mode** for Llama models—Llama-3.3-70B scored ~42.4% with prompt-based FC versus ~20.8% for native FC. [OpenReview](#) Databricks showed that with careful prompting and constrained decoding, Llama3-70B can surpass GPT-4o on BFCL. [Databricks](#)

Small models require constrained decoding, not better descriptions

For models under 70B, **structured output reliability is the binding constraint**, not description quality. vLLM's guided decoding (XGrammar backend) guarantees valid JSON schemas but cannot guarantee high-quality tool selection. [vLLM](#) The recommended stack: vLLM with `--enable-auto-tool-choice` and model-appropriate parser (`llama3_json` for Llama 3.1, `hermes` for Qwen 2.5+, [vLLM](#) `pythonic` for Llama 3.2+). [vLLM](#)

For 8B models, the Paragon study found that description enhancement had negligible effect on simple cases but improved multi-tool task completion from **37.5% to 53.9%** on complex multi-tool cases. [Paragon](#) The pattern is clear: invest in descriptions proportional to task complexity, not model capability. Small models benefit more from **fewer, better-described tools** than from many minimally described ones. [arXiv](#)

The comparison matrix

Dimension	Claude (4.x)	GPT-5 / o-series	Gemini (3/2.5)	Qwen3 / QwQ	DeepSeek R1	Llama 70B
Description style	3–4 sentences, extremely detailed	Prescriptive, key rules first	Clear and specific, use enums	Hermes-style with XML	N/A (no native FC)	1–2 sentences with formal examples
Tool count tolerance	50+ with Tool Search; 10–15 without	<100 in-distribution; 20–40 with clear descriptions	10–20 recommended; 128 hard limit	15–20 with Qwen-Agent	N/A	15–20
Parallel calling	Native; Sonnet most aggressive	API flag; disable for strict mode + fine-tuned models	Automatic; no config needed	Supported (Qwen3+)	N/A	Llama 4.0 not Llama 3.3
Prompt format	XML tags (recommended)	Markdown; developer role for o-series	XML or Markdown (both officially supported)	Flexible; Hermes template	User role only; no system prompt	Markdown Environment ipython header

Dimension	Claude (4.x)	GPT-5 / o-series	Gemini (3/2.5)	Qwen3 / QwQ	DeepSeek R1	Llama 70B
System prompt adherence	Very high (Opus overtriggers)	Surgical precision (GPT-5); reasoning models use developer role	High; Gemini 3 may over-analyze complex prompts	Good; thinking mode respects system	Ignores system prompt	Good at 70B; weak at 80B
Reasoning control	Extended thinking (token budget)	<code>reasoning_effort</code> : low/medium/high	<code>thinking_level</code> : low/high/minimal	<code>enable_thinking</code> , <code>/think</code> , <code>thinking_budget</code>	<code><think></code> tags; <code>max_tokens</code> only	None (no reasoning)
CoT in prompt?	Don't use "think"; use "consider"	Never for o-series; optional for GPT-5	Concise prompts for Gemini 3	<code>/think</code> toggle per turn	Never (model reasons internally)	Yes—helpful 70B
Temperature (tool calling)	Default	Default; reasoning handles internally	1.0 (Gemini 3); 0.0 (Gemini 2.5)	0.6 (thinking); 0.7 (chat)	0.6	0.0–0.3
Context window	200K (1M beta for Sonnet/Opus) <code>Claude API Docs</code>	1M+ (GPT-5); 200K (o-series)	1M (Gemini 2.5 Pro); varies	128K (Qwen3)	128K	128K
BFCL ranking	Top 3 (Opus 4.1: 70.36%) <code>KlavisAI</code>	GPT-5: 59.22% <code>KlavisAI</code>	Not in top 10	Qwen3-235B: 70.8% <code>arXiv</code>	Not ranked (no native FC)	42.4% (prompt-based)

Dynamic tool loading works—here is how to implement it

Three production-validated patterns exist for dynamic tool loading:

Anthropic's Tool Search Tool is the most mature. Mark tools with `defer_loading: true`; only the search tool itself (~500 tokens) loads initially. Claude discovers relevant tools via regex, BM25, or custom embedding search. `anthropic` Claude Code auto-enables this when MCP tool descriptions exceed 10% of the context window. `Anthropic` The implementation adds a single tool definition to your schema and a `defer_loading` boolean to existing tools.

The Meta-Tool Pattern (from Synaptic Labs and the Dynamic ReAct paper) replaces all tool schemas with two meta-tools: a discovery tool and an execution tool. Token overhead drops by **85–95%**. The LLM sees 2 tool registrations instead of 33+. Tools are grouped by "bounded context" `Synapticlabs` following the 7±2 cognitive science guideline.

LangGraph's embedding-based selection stores tool descriptions as vectors and retrieves the top-k most relevant tools per query. The LangGraph documentation provides a working example with 500+ tools (S&P 500 stock tools) reduced to 5 per query.

For your YAML-configured system, the Meta-Tool pattern offers the best portability across model families. It requires no vendor-specific API features—just two tools that any model can call. The Tool Search Tool is more accurate for Claude specifically but ties you to Anthropic's API extensions.

The minimal adaptation set that maximizes cross-model performance

Based on all research, a layered architecture with **four model-family adaptations** captures the majority of possible gains:

1. Format tags (highest impact). Use XML tags (`<instructions>`, `<context>`) for Claude. Use Markdown headers (`## Instructions`, `## Context`) for OpenAI models. Both formats work for Gemini and Qwen. A comparative study found XML consistently outperformed other formats even for Llama 3.1 405B, making XML a reasonable universal default if you want to maintain only one variant. [Robertodiasduarte](#)

2. Reasoning model detection (remove CoT instructions, adjust prompt length). For o-series, R1, and QwQ: remove all chain-of-thought prompting ("think step by step"), remove few-shot examples, and shorten prescriptive instructions. [Helicone +2](#) For R1 specifically: move all instructions from system prompt to user role. [Data Studios](#) [together](#) Your existing `Reasoning: high` tag concept is on the right track—extend it to trigger these removals.

3. Parallel tool calling guidance (explicit prompt instruction). Every major coding agent (Claude Code, Cursor, Kimi CLI) includes explicit parallel calling instructions because **models default to serial calls** due to training data bias. [Drew Breunig](#) Add a format-appropriate block instructing the model to parallelize independent tool calls. Disable parallel calling for Llama 3, Mistral 7B, [vLLM](#) and gpt-4.1-nano. [OpenAI](#) This is a prompt-level concern beyond just the API flag.

4. Tool set sizing per model class. Implement dynamic tool loading with model-aware limits: present all tools for Claude Opus/Sonnet (handles 50+ well), [anthropic](#) limit to 20 for GPT-5/o-series [openai](#) and Gemini, limit to 10–15 for Qwen/Llama 70B, and limit to 5–8 for any model under 70B. Your two-tier description system (full docstrings for core tools, 1-line for domain tools) aligns well with the Tool Search pattern—short descriptions serve as search-optimized summaries while full docs load on demand.

A recommended YAML schema for model classification:

```
yaml
```

```

model_classes:
  frontier_reasoning:
    patterns: ["claude-opus-*", "o3-*", "o4-*", "gpt-5*"]
    tool_limit: 40
    parallel_tools: true
    format: model_native # XML for Claude, Markdown for OpenAI
    cot_instructions: remove_for_o_series

  standard:
    patterns: ["claude-sonnet-*", "claude-haiku-*", "gpt-4o*", "gemini-*"]
    tool_limit: 20
    parallel_tools: true
    format: model_native
    cot_instructions: keep

  oss_reasoning:
    patterns: ["deepseek-r1*", "qwq-*", "qwen*-thinking*"]
    tool_limit: 15
    parallel_tools: model_dependent
    format: xml # Works well across all OSS models
    cot_instructions: remove
    system_prompt: user_role_for_r1

  compact:
    patterns: ["llama-*-8b*", "qwen*-7b*", "qwen*-14b*"]
    tool_limit: 8
    parallel_tools: false
    format: xml
    cot_instructions: keep_explicit
    constrained_decoding: required

```

Conclusion

The research converges on a counterintuitive finding: **universal prompts with runtime tool management outperform model-specific prompt variants** for most practical purposes. The performance gap between a well-tuned Claude prompt and a generic prompt running on Claude is far smaller than the gap between giving Claude 40 tools versus 8 relevant tools. Invest engineering effort in dynamic tool loading and model-class-based tool limits first, format adaptation second, and full prompt variants only for the highest-stakes agent configurations. (Paragon)

Three specific changes for your system ordered by expected impact: First, implement tool search with (defer_loading) for your domain tools (your existing 1-line short descriptions are already ideal search summaries). Second, add a (model_class) field to your YAML config with four categories (frontier_reasoning, standard, oss_reasoning, compact) that controls tool limits, parallel calling, and format selection at runtime. Third, add a conditional block that strips CoT instructions and few-shot examples when (model_class) is

`oss_reasoning`—this alone can prevent the "asking a reasoning model to reason more hurts performance" failure mode `Helicone +2` that is likely your largest current source of suboptimal behavior when switching from chat models to R1 or QwQ.